

My Places – Le journal intime géographique



- **Technologies imposées** : Java / Kotlin, Android Studio, Jetpack Compose, Room (SQLite), Retrofit, Google Maps SDK.

Description du projet

L'objectif est de réaliser **My Places**, une application Android native faisant office de journal de bord intime et géographique. L'application permet à l'utilisateur de cartographier ses souvenirs, ses découvertes ou ses humeurs en "taggant" des lieux précis sur une carte ou par géolocalisation. Chaque lieu est enrichi d'un texte, d'un emoji représentant un sentiment ou une catégorie, et d'une photo.

L'application devra impérativement gérer la persistance locale avancée, la consommation d'un web service asynchrone, et proposer une fonctionnalité d'import/export de données permettant de fusionner les souvenirs d'un autre utilisateur.

Fonctionnalités attendues (Cahier des charges)

1. Cartographie et exploration (Écran principal)

- **Intégration Google Maps** : Affichage d'une carte plein écran centrée par défaut sur la position actuelle de l'utilisateur (permissions ACCESS_FINE_LOCATION à gérer proprement).
- **Marqueurs personnalisés** : Chaque lieu enregistré doit apparaître sur la carte sous forme de marqueur. L'icône du marqueur doit afficher dynamiquement l'emoji choisi par l'utilisateur.
- **Consultation** : Un clic sur un marqueur ouvre une fiche détaillée (ou une bottom sheet) affichant le titre, la date, l'adresse textuelle, le ressenti et la photo associée.

2. Enregistrement d'un nouveau lieu

- Un clic long sur la carte ou un bouton d'action permet de capturer les coordonnées GPS du point visé et ouvre un formulaire d'ajout.
- **Le formulaire doit comporter** :
 - Un titre et une description textuelle.
 - Un sélecteur d'emoji (ex: 🚀, ❤️, 🍕, 🌲).
 - Une capture photo (via l'appareil photo avec l'API **CameraX** ou via la galerie interne).
- **Reverse Geocoding (Appel API REST)** : Lors de la validation, l'application ne doit pas simplement stocker la latitude et la longitude. Elle doit interroger en arrière-plan un web service de géocodage inverse (ex: l'API publique et gratuite adresse.data.gouv.fr) pour récupérer l'adresse postale réelle du lieu et l'associer au souvenir.

3. Persistance des données et architecture

- Toutes les données doivent être persistées localement dans une base de données SQLite via **Room**.
- **Gestion des fichiers** : Interdiction stricte de stocker les photos sous forme de chaînes Base64 dans la base de données. Les images doivent être sauvegardées dans le stockage interne privé de l'application, et seul leur chemin d'accès (URI ou String) doit être enregistré dans Room.

4. Partage et évolution de la base (Import / Export JSON)

- L'application doit proposer une fonctionnalité d'**Export** permettant de sérialiser l'intégralité du journal au format `places_export.json`.
- Elle doit également proposer une fonctionnalité d'**Import** permettant de lire un fichier JSON fourni par un ami pour intégrer ses lieux partagés dans sa propre application.
- **Contrainte de modélisation (Attention)** : Pour que l'import fonctionne sans écraser ou corrompre vos propres données, votre base de données locale doit être capable de différencier vos propres lieux de ceux importés depuis le fichier d'un ami (notion d'utilisateur/auteur à anticiper dès la conception du schéma de la base de données).

5. Sécurité

- S'agissant d'un journal intime, l'application doit intégrer un verrouillage biométrique optionnel (API BiometricPrompt). Si activé, l'accès à l'application requiert la validation de l'empreinte digitale ou du schéma du smartphone au démarrage.

Livrables attendus

1. Le lien vers un dépôt Git public (GitHub / GitLab) contenant l'historique des commits (l'équilibre du travail entre les membres du groupe sera vérifié via les commits).
2. Un court fichier README.md à la racine expliquant les choix techniques, l'architecture choisie, et le format du fichier JSON d'échange.
3. Une démonstration live (sur émulateur ou device physique) lors de la soutenance.